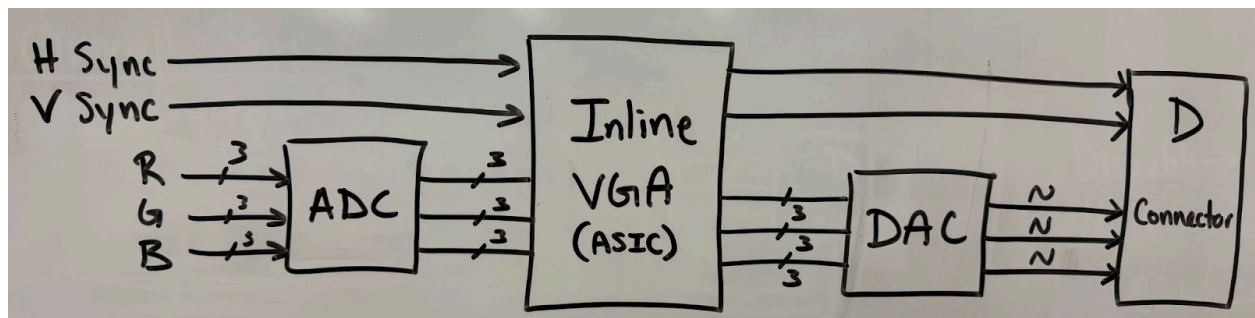


18-624 Project Proposal

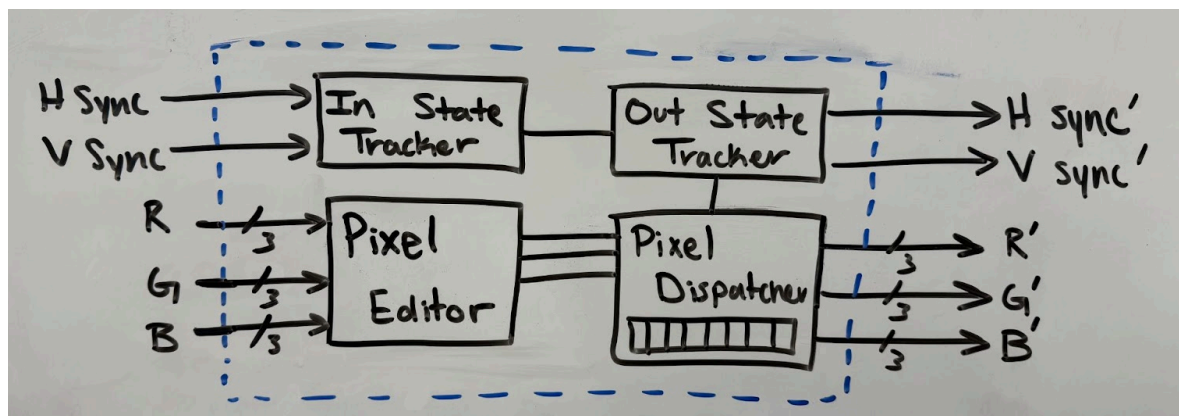
Introduction

InlineVGA is an ASIC that receives image data in VGA format. The data is modified (exact transformations not decided) and then passed to a VGA monitor as the output. The ASIC will hopefully work in a relatively streaming fashion with minimal data buffering, due to resource constraints. The circuit will be set “inline” with input coming directly from a VGA cable and the output going to the D connector of a VGA monitor.

Overall System Design



InlineVGA Block Design



*Reset signal not pictured

Modules

Incoming State Tracker

This module will handle tracking all the state information from the incoming VGA cable. This included the pixel row and column, and most importantly horizontal and vertical syncs. The module will communicate with the Outgoing State Tracker. It will receive inputs from incoming VGA horizontal and vertical syncs and will output to the Outgoing State Tracker.

Outgoing State Tracker

Similar to the Incoming Tracker, this will track the state of data that is being passed to the outgoing VGA cable. It includes the outgoing version of the information mentioned above. In addition, it will control the queueing logic of the pixel dispatcher. There will be moments where the outgoing VGA is in the front or back porch and not consuming the pixel data passed by the incoming VGA. During these periods the incoming pixel data will need to be buffered until it is ready to be dispatched. The module will communicate with Pixel Dispatcher and D connector directly. It will receive inputs from the Incoming State Tracker and output to the Pixel Dispatcher, outgoing horizontal and vertical syncs, and an error signal.

Pixel Editor

The editor is realistically going to be a wrapper around all the different pixel transformations I have time to implement. If all goes well, I plan to have monochromatic filters for RGB and averaging. The baseline will be swapping RGB values as they come into the ASIC. The inputs of the module will be the digital version of the incoming RGB values and the output will be the Pixel Dispatcher.

Pixel Dispatcher

This will handle passing the modified pixel values to DAC and eventually the outgoing VGA cable. Additionally, the module is responsible for queueing RGB values when the outgoing VGA is not consuming pixel data (i.e front and back porch), but the incoming VGA is producing pixel data. The module will take input from the Pixel Editor and the Outgoing State Tracker and will output the ongoing RGB values to DAC and eventually the VGA cable.

Inputs/Outputs

Input	Width	Description
Reset	1	Reset for all modules
Red	3	Red channel data after ADC

Green	3	Green channel data after ADC
Blue	3	Blue channel data after ADC
H Sync	1	Horizontal sync pulse from VGA cable
V Sync	1	Vertical sync pulse from VGA cable
Total	12	N/A

Output	Width	Description
Red	3	Red channel data before DAC
Green	3	Green channel data before DAC
Blue	3	Blue channel data before DAC
H Sync	1	Horizontal sync pulse from VGA cable
V Sync	1	Vertical sync pulse from VGA cable
Error	1	Error if something goes wrong
Total	12	N/A

Additional Hardware

Hardware	Quantity	Description
Pmod VGA	x2	Intercepting and Outputting VGA signals
*DAC	x1	Converting digital RGB values to analog value for VGA
*ADC	x1	Converting analog RGB values into digital format

* Need to look into more to find out exactly what fits best for this scenario

Open Questions

1. Is the synchronization between the incoming VGA cable and the ASIC as simple as waiting for the horizontal sync drop, and then tracking state from that point on?
2. How to map incoming and outgoing horizontal and vertical syncs?
3. How to handle buffering pixel data when incoming and outgoing aren't producing and consuming at the same time?
4. Does the module need to be able to handle variable image sizes?

5. What refresh rate to target? Standard is 60Hz, but that requires a ~25MHz clock.

Next Steps

1. Determining a target refresh rate for the ASIC
2. Calculating values for VGA control delays
3. Design a FSM to track incoming VGA state
4. Design a FSM to track outgoing VGA state
5. Design a Pixel Dispatcher with buffer logic
6. Design a Pixel Editor that switches RGB values of input